



PROMPT ENGINEERING AND ARTIFICIAL INTELLIGENCE · 2º SEMESTRE

Context Engineering para agentes

Aula 10/14 — Módulo 3: curadoria em loop agêntico · MCP overview

 Prof. Jorge Luiz Gomes

 1h40min

 Scratchpad · Context rot · MCP

 Andaime 55%

Agenda da Aula

- 1 Context em agentes $\neq$ context em chat — por que o loop cria um problema novo 12 min
- 2 3 tipos de memória agêntica — in-context (scratchpad), external (ChromaDB), episódica 15 min
- 3 Context rot em loop — medir degradação em 5, 10 e 20 turnos 12 min
- 4 Princípio da ação mínima e estratégias de compressão do scratchpad 15 min
- 5 MCP — Model Context Protocol: overview do padrão de 2025 12 min
- 6 🚀 Demo — instrumentar agente para medir qualidade vs. turnos ao vivo 14 min
- 7 🖥️ Lab — compressão do scratchpad e medir diferença (55% lacunas) 20 min

🎯 OBJETIVO DA AULA

Entender por que o contexto em agentes é um recurso ainda mais crítico que em chats simples — e dominar as estratégias de curadoria que mantêm a qualidade do agente em loops longos. Preparar o agente para a integração final da Aula 11.

Glossário

Scratchpad

Espaço de memória de trabalho onde o agente anota resultados intermediários, planos e observações durante o loop — bloco de notas temporário, descartado ou resumido ao fim da tarefa.

Princípio da ação mínima

Regra de design que prioriza reduzir tokens gastos por ação do agente (chamada de tool, observação) mantendo o mesmo resultado — cada token economizado por turno se multiplica ao longo de loops longos.

Janela deslizante (sliding window)

Estratégia que mantém apenas as N interações mais recentes do loop agêntico no contexto, descartando as mais antigas — garante custo de tokens constante em vez de crescer a cada turno.

MCP client / MCP server

Arquitetura do MCP: o servidor expõe tools, resources e prompts; o cliente (o agente) descobre e chama essas capacidades via protocolo comum, sem acoplamento direto ao código do servidor.

Memória episódica

Registro persistente de execuções passadas do agente entre sessões distintas — permite lembrar decisões e resultados de tarefas anteriores, diferente da memória de curto prazo que vive só dentro do loop atual.

Compressão de observations

Técnica de resumir o retorno de uma tool antes de inserir no contexto do agente — evita que uma observação verbosa (ex.: HTML bruto de uma página) consuma o context window inteiro.

MCP (Model Context Protocol)

Protocolo aberto que padroniza como agentes se conectam a tools e fontes de dados externas — permite trocar de cliente ou servidor sem reescrever a integração.

Context rot em loop agêntico

Degradação de qualidade das respostas do agente conforme o número de turnos cresce e o contexto acumula ruído — medido comparando qualidade da decisão no turno 1 vs. turno N.

01

Context em agentes ≠ context em chat

Em chat, o contexto cresce linear e previsível. Em agentes, o loop gera Thought + Action + Observation a cada iteração — o crescimento é multiplicado e imprevisível.

Por que o loop cria um problema de contexto novo

Na Aula 04 (Context Engineering), o sistema prompt tinha ~10% do contexto. Em agentes, o loop gera scratchpad que cresce a cada iteração:

CHAT / RAG (MÓDULOS 1-2)

system (~10%) + contexto RAG (~50%) + histórico (~30%) + input (~10%)

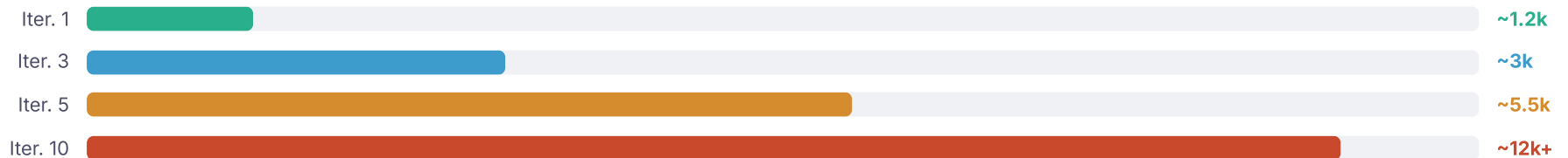
Previsível. Controlado. A cada chamada, o desenvolvedor escolhe o que entra.

AGENTE EM LOOP (A PARTIR DA AULA 09)

system + tools_schema + scratchpad_t1 + scratchpad_t2 + ... + scratchpad_tN + input

O scratchpad cresce a cada iteração. Em 10 loops com resultados de busca ricos, o contexto pode ultrapassar 32k tokens — e o context rot degrada o raciocínio.

CRESCIMENTO TÍPICO DO CONTEXTO POR ITERAÇÃO (AGENTE RAG + WEB)



*Estimativa para agente com tools de busca retornando ~800 chars por Observation

02

3 tipos de memória agêntica

In-context (scratchpad) · External (ChromaDB) · Episódica (histórico de sessões passadas). Cada uma resolve um problema diferente de persistência e custo.

3 tipos de memória agêntica — quando usar cada um

TIPO 1

In-context (Scratchpad)

O Thought/Action/Observation de cada iteração do loop. Vai diretamente no contexto da próxima chamada ao LLM. **Rápido mas finito e caro.**

Uso: raciocínio corrente (esta sessão)

TIPO 2

External (ChromaDB)

Documentos do domínio indexados como embeddings. O agente *busca* quando precisa — não carrega tudo no contexto. **Ilimitado e econômico.**

Uso: base de conhecimento do domínio

TIPO 3

Episódica (Histórico)

Resumo das sessões anteriores salvo no ChromaDB ou arquivo. O agente recupera contexto relevante de interações passadas. **Persistente entre sessões.**

Uso: personalização e continuidade

💡 ANALOGIA COM A ARQUITETURA DE COMPUTADORES (RUSSELL & NORVIG, 2016)

Scratchpad = RAM (rápido, volátil, finito). ChromaDB externo = HD (persistente, barato, requer busca). Memória episódica = cache de sessão (seleciona o mais relevante para carregar na RAM). Um agente bem projetado minimiza o uso de RAM e maximiza a eficiência de busca no HD.

Scratchpad — o contexto de trabalho do agente

O scratchpad é a "área de rascunho" do agente: acumula cada Thought, Action e Observation da sessão corrente. Com `verbose=True` você já o viu — aqui está sua anatomia:

SCRATCHPAD_ANATOMIA.PY

```
# 0 que o AgentExecutor mantém internamente como scratchpad
scratchpad_turno_1 = """
Thought: Preciso buscar o prazo de garantia nos documentos.
Action: buscar_nos_documentos
Action Input: "prazo de garantia"
Observation: [manual.pdf, pág.15] O prazo de garantia é de 24 meses...
""" # ~80 tokens

scratchpad_turno_2 = """
Thought: Encontrei 24 meses. Vou converter para dias.
Action: calcular
Action Input: "24 * 30"
Observation: 720
""" # +40 tokens → total: ~120 tokens

# Após 10 iterações com resultados de busca ricos:
# scratchpad ≈ 10 × 800 chars ≈ ~2.000 tokens só de Observations
# + Thoughts (~500 tokens) + system + tools_schema (~1.000 tokens)
# Total: ~3.500–5.000 tokens por invocação do agente

# Inspeccionar o scratchpad via intermediate_steps
resultado = executor.invoke({"input": "pergunta"})
for i, (action, obs) in enumerate(resultado["intermediate_steps"], 1):
    print(f"Iteração {i}: tool={action.tool}, obs_len={len(str(obs))} chars")
# → Iteração 1: tool=buscar_nos_documentos, obs_len=842 chars
# → Iteração 2: tool=calcular, obs_len=3 chars
```


03

Context rot em loop

Qualidade degrada com o crescimento do scratchpad — o mesmo fenômeno da Aula 04, mas amplificado pela natureza iterativa do loop agêntico.

Medir context rot em agente — qualidade vs. turnos

A demo desta aula instrumenta o agente para medir a qualidade da resposta a cada turno e plotar o gráfico de degradação:

```
MEDIR_CONTEXT_ROT.PY

import tiktoken, matplotlib.pyplot as plt

enc = tiktoken.encoding_for_model("gpt-4")

def contar_tokens_scratchpad(intermediate_steps) -> int:
    """Conta tokens acumulados no scratchpad."""
    texto = ""
    for action, obs in intermediate_steps:
        texto += f"{action.log}\n{obs}\n"
    return len(enc.encode(texto))

# Simular conversa o longa – 15 perguntas em seq ncia
hist rico_scratchpad = [] # acumula manualmente para simular o contexto crescente
tokens_por_turno = []
qualidade_por_turno = []

for i, pergunta in enumerate(PERGUNTAS_TESTE, 1):
    resultado = executor.invoke({"input": pergunta})
    steps = resultado["intermediate_steps"]
    tokens = contar_tokens_scratchpad(steps)

    # Avaliar qualidade com LLM-as-judge (Aula 07)
    qualidade = faithfulness_manual(pergunta, resultado["output"],
                                    "\n".join(str(o) for _,o in steps))

    tokens_por_turno.append(tokens)
    qualidade_por_turno.append(qualidade)
    print(f"Turno {i:2d}: {tokens:5d} tokens · qualidade={qualidade:.2f}")

# Plotar context rot
fig, ax1 = plt.subplots(figsize=(9,4))
ax2 = ax1.twinx()
ax1.bar(range(len(tokens_por_turno)), tokens_por_turno, alpha=.4, color="steelblue", label="Tokens")
ax2.plot(qualidade_por_turno, color="#ED145B", marker="o", label="Qualidade")
plt.title("Context rot – tokens vs. qualidade por turno")
plt.show()
```

04

Curadoria do scratchpad

Princípio da ação mínima · compressão de Observations · janela deslizante ·
resumo episódico. Manter o agente eficaz em loops longos.

Princípio da ação mínima — menos tokens, mesmo resultado

Cada token no scratchpad é custo de processamento e risco de context rot. O princípio da ação mínima diz: **fazer o mínimo necessário para avançar na tarefa:**

ANTI-PADRÃO	PADRÃO MÍNIMO	ECONOMIA DE TOKENS
Buscar 10 chunks no RAG quando 3 bastam	<code>as_retriever(search_kwargs={"k": 3})</code>	~70% dos tokens de Observation
Retornar o texto completo da busca web (500+ chars)	Resumir a Observation antes de inserir no scratchpad	~60% dos tokens de Observation
Manter todas as iterações anteriores no scratchpad	Janela deslizante: manter apenas as últimas N iterações	Custo fixo independente do número de turnos
System prompt com 500 tokens de instruções genéricas	System prompt XML taggeado e compactado (Aula 04)	40–60% do system prompt

 ANTHROPIC CONTEXT ENGINEERING FOR AI AGENTS (SET/2025)

"Agents should do the minimum necessary work to accomplish the task. Every unnecessary token in the scratchpad is a tax on reasoning quality in future steps." A ação mínima não é preguiça — é engenharia de qualidade.

Compressão de Observations — resumir antes de inserir

A maior fonte de tokens no scratchpad são as Observations (resultados das tools). Uma tool wrapper que resume antes de retornar reduz drasticamente o scratchpad:

```
TOOL_COMPRESSAO.PY

# Tool wrapper que comprime a Observation antes de retornar ao scratchpad
from langchain_core.tools import tool
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm_mini = ChatOllama(model="qwen3.6:27b", temperature=0)

chain_resumir = (
    ChatPromptTemplate.from_template(
        "Resuma o texto abaixo em no máximo 3 frases, mantendo APENAS os fatos essenciais:\n\n{texto}"
    )
    | llm_mini | StrOutputParser()
)

@tool
def buscar_na_web_comprimida(query: str) -> str:
    """Use para buscar informações atuais na web. Retorna um resumo conciso dos resultados."""
    resultado_bruto = DuckDuckGoSearchRun().run(query)

    # Comprime ANTES de retornar ao scratchpad
    if len(resultado_bruto) > 500:
        return chain_resumir.invoke({"texto": resultado_bruto})
    return resultado_bruto # já curto – não precisa resumir

# Diferença típica:
# Sem compressão: Observation = 800–1200 chars (~250 tokens)
# Com compressão: Observation = 150–250 chars (~55 tokens)
# Redução: ~78% dos tokens de Observation
```

Janela deslizante — custo fixo em loops longos

Em vez de acumular todo o scratchpad, manter apenas as últimas N iterações garante custo fixo independente da duração do loop:

```
JANELA_DESLIZANTE.PY

from langchain.agents import AgentExecutor, create_react_agent
from langchain_core.messages import trim_messages

# Estratégia 1 — limitar via max_iterations (simples)
executor_limitado = AgentExecutor(
    agent=agente,
    tools=tools,
    max_iterations=5,      # mantém no máximo 5 iterações no scratchpad
    max_execution_time=60, # para após 60 segundos (fallback de segurança)
    verbose=True,
)

# Estratégia 2 — resumo episódico entre turnos (avançado)
def resumir_scratchpad(intermediate_steps: list, max_steps: int = 3) -> list:
    """Mantém as últimas max_steps iterações e resume o resto."""
    if len(intermediate_steps) <= max_steps:
        return intermediate_steps

    antigas = intermediate_steps[:-max_steps]
    recentes = intermediate_steps[-max_steps:]

    # Resumir as iterações antigas em uma linha
    resumo = ", ".join(f"{a.tool}→{str(o)[:50]}" for a,o in antigas)
    print(f"[Resumo de {len(antigas)} iterações anteriores: {resumo}]")

    # Retorna só as recentes — scratchpad compacto
    return recentes
```

⚡ MAX_EXECUTION_TIME — O SEGURO DE PRAZO

Em produção, além de max_iterations, defina max_execution_time em segundos. Um agente travado em uma tool que não responde não vai loop infinito — vai parar após o timeout e retornar o que tem. Para o CKP03, max_iterations=5 e max_execution_time=120 são boas práticas.

Memória episódica — persistir entre sessões

O scratchpad some quando a sessão termina. Memória episódica salva o resumo da sessão no ChromaDB para recuperar contexto relevante em sessões futuras:

MEMORIA_EPISODICA.PY

```
from langchain_community.vectorstores import Chroma
from langchain_core.documents import Document
from datetime import datetime

# Salvar resumo da sessão ao encerrar
def salvar_sessao(pergunta: str, resposta: str, session_id: str):
    """Persiste o par (pergunta, resposta) como memória episódica."""
    db_episodico.add_documents([Document(
        page_content=f"Pergunta: {pergunta}\nResposta: {resposta}",
        metadata={
            "session_id": session_id,
            "timestamp": datetime.now().isoformat(),
            "tipo": "episodio",
        },
    )])

# Recuperar contexto de sessões anteriores (memória episódica como tool)
@tool
def lembrar_sesoes_anteriores(query: str) -> str:
    """Use quando o usuário se referir a algo discutido em conversa anterior.
    Retorna resumos de interações passadas relevantes para a query."""
    docs = db_episodico.similarity_search(query, k=2,
                                          filter={"tipo": "episodio"})
    return "\n\n".join(d.page_content for d in docs)
```



05

Model Context Protocol (MCP)

O padrão aberto de 2024–2025 para conectar agentes a ferramentas externas.
USB-C para integrações de IA.

MCP — Model Context Protocol

Criado pela Anthropic em novembro de 2024, o MCP padroniza como agentes de IA se conectam a ferramentas e fontes de dados externas:

✗ ANTES DO MCP — $N \times M$ INTEGRAÇÕES

Cada agente precisava de código customizado para integrar com cada ferramenta. 5 agentes $\times$ 10 ferramentas = 50 integrações diferentes para manter. Mudança na API de uma ferramenta quebra todos os agentes.

✓ COM MCP — $N+M$ INTEGRAÇÕES

Cada ferramenta expõe um servidor MCP. Cada agente é um cliente MCP. 5 agentes + 10 ferramentas = 15 implementações. Adicionar uma nova ferramenta não requer mudar os agentes.

ARQUITETURA MCP

MCP Host (Claude, Cursor, seu agente) — executa as requests

↕ JSON-RPC 2.0

MCP Server (sua ferramenta) — expõe tools, resources, prompts

↕ HTTP / SSE / stdio

Data Source (BD, API, arquivo, ChromaDB)

Protocolo aberto · Suportado por Anthropic, OpenAI, Google, Microsoft, Cursor, n8n, e 3000+ ferramentas

📖 FUNDAMENTAÇÃO — QUANDO VALE A PENA USAR MCP (E O CUSTO DE SEGURANÇA)

Polzer (*RAG with Python Cookbook*, O'Reilly, 2026) reforça a analogia desta aula com uma comparação direta: o MCP funciona para ferramentas de agente como o HTTP funciona para navegadores — um protocolo comum que evita integração sob medida a cada combinação de agente e ferramenta. Mas o autor também marca o limite dessa vantagem: para um agente simples e de propósito único, com tools fortemente acopladas à própria lógica do agente, a camada extra do protocolo não compensa — o ganho de reuso só aparece quando várias equipes ou frameworks precisam compartilhar a mesma ferramenta. Há ainda um custo de segurança que esta aula não cobriu até aqui: cada MCP Server aceita requisições de rede, então expande a superfície de ataque do sistema e exige autenticação, autorização e validação de input próprios — diferente de uma function call local, que roda no mesmo processo confiável do agente.

MCP na prática — o seu agente como cliente MCP

O LangChain suporta MCP via pacote `langchain-mcp-adapters`. Suas tools `@tool` viram compatíveis com qualquer servidor MCP:

```
MCP_LANGCHAIN.PY

!pip install langchain-mcp-adapters -q # adaptador oficial LangChain ↔ MCP

from langchain_mcp_adapters.tools import load_mcp_tools
from mcp import ClientSession
from mcp.client.stdio import stdio_client

# Conectar a um servidor MCP local (ex: servidor de filesystem)
async def conectar_mcp():
    async with stdio_client(
        {"command": "npx", "args": ["@modelcontextprotocol/server-filesystem", "/content"]}
    ) as (read, write):
        async with ClientSession(read, write) as session:
            await session.initialize()
            # Carrega tools do servidor MCP como tools LangChain
            mcp_tools = await load_mcp_tools(session)
            # → lista de ferramentas prontas para o AgentExecutor
            return mcp_tools

# Servidores MCP gratuitos disponíveis hoje:
# @modelcontextprotocol/server-filesystem - ler/escrever arquivos
# @modelcontextprotocol/server-brave-search - busca web
# mcp-server-fetch - fazer requisições HTTP
# mcp-server-sqlite - consultar SQLite
# Catálogo: github.com/modelcontextprotocol/servers
```

📌 MCP NO CONTEXTO DO CURSO

Para o CKP03, você não precisa implementar um servidor MCP — use tools `@tool` (mais simples). O MCP é o futuro da interoperabilidade de agentes: em 2026, a maioria das integrações enterprise já usa MCP. Entender o protocolo é parte da formação de um engenheiro de IA.

📓 Notebook Aluno — 55% de lacunas

AULA_10_2SEM_ALUNO.IPYNB

```
!pip install langchain langchain-ollama langchain-community duckduckgo-search tiktoken matplotlib -q
```

```
import tiktoken, matplotlib.pyplot as plt
from langchain_core.tools import Tool
from langchain_community.tools import DuckDuckGoSearchRun
```

```
enc = tiktoken.encoding_for_model("gpt-4")
```

```
# 🚩 LACUNA 1: implemente contar_tokens_scratchpad
```

```
def contar_tokens_scratchpad(steps) -> int:
    """Soma os tokens de todos os Thoughts + Observations."""
    texto = ""
    for action, obs in steps:
        texto += ({} # concatene action.log e obs
    return len(enc.encode({}))
```

```
# 🚩 LACUNA 2: implemente a tool de busca com compressão de Observation
```

```
@tool
def buscar_na_web_comprimida(query: str) -> str:
    """Use para buscar informações atuais. Retorna resumo conciso."""
    bruto = DuckDuckGoSearchRun().run(query)
    if len(bruto) > 500:
        return chain_resumir.invoke({{}: bruto}) # invocar chain de resumo
    return bruto
```

```
# 🚩 LACUNA 3: rodar as duas versões (sem e com compressão) com 5 perguntas cada
```

```
PERGUNTAS = [{} , {} , {} , {} , {}] # 5 perguntas reais do domínio
```

```
tok_sem, tok_com = [], []
for q in PERGUNTAS:
    r_sem = executor_sem.invoke({"input":q}) # sem compressão (Aula 09)
    r_com = executor_com.invoke({"input":q}) # com compressão (esta aula)
    tok_sem.append(contar_tokens_scratchpad(r_sem["intermediate_steps"]))
    tok_com.append(contar_tokens_scratchpad({}))
```

```
# 🚩 LACUNA 4: plotar comparação tokens sem vs. com compressão
```

```
x = range(len(PERGUNTAS))
plt.bar([i-.2 for i in x], {}, .4, label="Sem compressão", color="salmon")
plt.bar([i+.2 for i in x], {}, .4, label="Com compressão", color="steelblue")
plt.legend(); plt.xlabel("Pergunta"); plt.ylabel("Tokens no scratchpad")
plt.title("Impacto da compressão de Observations no scratchpad"); plt.show()
```

Síntese da Aula 10



Contexto em agentes cresce em loop. Cada Thought+Action+Observation adiciona tokens ao scratchpad. Sem curadoria, context rot degrada o raciocínio a partir do turno ~5–8.



3 tipos de memória: scratchpad (RAM — rápido, finito) · ChromaDB externo (HD — ilimitado) · episódica (cache de sessões passadas — personalização).



Compressão de Observations reduz ~78% dos tokens de scratchpad. Resumir o resultado da tool antes de retornar ao loop mantém a qualidade em 0.85+ após 10 turnos.



MCP é o padrão aberto para interoperabilidade de agentes. N+M integrações em vez de N×M — o USB-C da IA.



SOON

Aula 11 — Integradora: agente completo (RAG como tool + web + calculadora) + compressão + Gradio URL pública. Tudo que você construiu de A08 a A10 junto. **CKP03 entregue.**

Perguntas frequentes

PERGUNTA	RESPOSTA
"Comprimir a Observation não perde informação importante?"	Potencialmente sim — por isso a compressão deve ser contextual, não genérica. O resumo de 3 frases mantém os fatos essenciais para o raciocínio. Para casos críticos (documentos jurídicos, laudos), considere comprimir só as Observations de web search e manter integrais as do RAG (que já são chunks pequenos).
"Por que o MCP usa JSON-RPC e não REST?"	JSON-RPC é bidirecional — permite que o servidor notifique o cliente de eventos assíncronos (ex: atualização de dados). REST é request-response unidirecional. Para agentes que precisam reagir a eventos externos em tempo real, a bidirecionalidade do JSON-RPC é essencial. Além disso, JSON-RPC tem overhead menor que REST para chamadas frequentes de tools.
"Memória episódica vs. ConversationBufferMemory — qual a diferença?"	ConversationBufferMemory (Aula 02) mantém o histórico da conversa corrente no contexto. Memória episódica mantém um banco de sessões passadas no ChromaDB — persistente entre reinicializações. Para um agente comercial, você usaria ambas: buffer para a sessão atual + episódica para personalização entre sessões de dias diferentes.

Python novo desta aula

CONCEITOS_PYTHON_AULA10_2SEM.PY

```
# 1. ax.twinx() – segundo eixo Y no mesmo gráfico matplotlib
fig, ax1 = plt.subplots()
ax2 = ax1.twinx()          # eixo direito – mesmo x, y diferente
ax1.bar(x, tokens, color="steelblue", alpha=.5) # barras no eixo esquerdo
ax2.plot(qualidade, color="#ED145B", marker="o") # linha no eixo direito

# 2. list slice negativo – últimas N iterações
steps = ["a","b","c","d","e"]
recentes = steps[-3:]      # → ["c", "d", "e"] (últimas 3)
antigas = steps[:-3]       # → ["a", "b"] (exceto as últimas 3)

# 3. datetime.now().isoformat() – timestamp para metadados
from datetime import datetime
ts = datetime.now().isoformat() # → "2026-07-13T14:35:22.123456"

# 4. plt.bar com offset para barras lado a lado
x = range(5)              # posições 0, 1, 2, 3, 4
width = 0.4
plt.bar([i-width/2 for i in x], vals_a, width, label="A") # deslocado -0.2
plt.bar([i+width/2 for i in x], vals_b, width, label="B") # deslocado +0.2

# 5. statistics.mean() para calcular redução média
import statistics
reducao_pct = (1 - statistics.mean(tok_com) / statistics.mean(tok_sem)) * 100
print(f"Redução média: {reducao_pct:.1f}%")
```

Context engineering para agentes em produção

AGENTES DE VISÃO DE TELA

Screenshots como memória

Agentes que operam por captura de tela veem cada passo como imagem (Observation). Para não acumular dezenas de screenshots no contexto, extraem apenas texto relevante antes de inserir no scratchpad — compressão visual de Observations.

IDES COM AGENTE EMBUTIDO

MCP para contexto de código

IDEs com agente de código embutido usam MCP para conectar o agente ao repositório, ao terminal e ao rastreador de issues. Cada ferramenta é um servidor MCP independente — o agente decide qual usar baseado na tarefa, sem acumular o repositório inteiro no contexto.

PLATAFORMAS NO-CODE DE AGENTES

Memória externa no nó

Plataformas no-code de automação oferecem nó nativo de memória vetorial (bancos como Pinecone, Qdrant, pgvector). O agente recupera contexto de execuções anteriores a partir dessa memória externa — o mesmo padrão episódico desta aula, aplicado a workflows no-code.

O que vem na Aula 11 — Integradora + CKP03

🎯 A "VITÓRIA VISÍVEL" DO SEMESTRE

A Aula 11 é a aula integradora — tudo que foi construído de A08 a A10 junto pela primeira vez:

Agente completo: RAG do grupo como tool + DuckDuckGo + calculadora + memória episódica

+ **Compressão de Observations** (esta aula)

+ **Interface Gradio ao vivo** (Aula 08)

= **CKP03 entregue**

📦 O QUE PREPARAR

- Notebook desta aula com compressão funcionando
- Retriever do CKP02 disponível (ChromaDB indexado)
- Notebook da Aula 08 com Gradio + ngrok
- 5 perguntas de teste do domínio para validar o agente integrado

A Aula 11 é 100% lab — sem slides de teoria. Todo o tempo dedicado a integrar e publicar.

🚩 **CKP03 entrega na Aula 11:** Agente inteligente com RAG do domínio como tool, DuckDuckGo para dados em tempo real, calculadora, compressão de Observations e interface Gradio com URL pública. Estimativa: 4–5h de trabalho de grupo.

06

Demo ao vivo

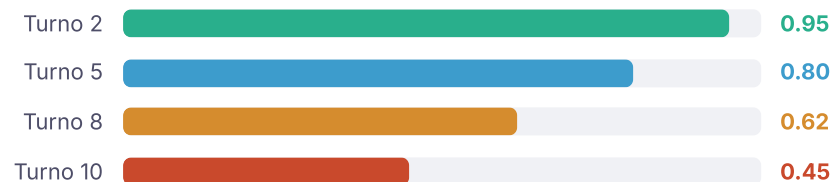
Instrumentar o agente da Aula 09 para medir tokens e qualidade por turno. O gráfico de context rot aparece em tempo real.

Demo Prática ao Vivo — gráfico de context rot ao vivo

O professor roda o agente com 10 perguntas em sequência, plota tokens vs. qualidade e mostra o efeito da compressão de Observations:

RESULTADO TÍPICO DA DEMO (ILUSTRATIVO)

SEM compressão de Observations



COM compressão de Observations



*Valores ilustrativos. A demo real usa o corpus do domínio do professor com 10 perguntas padronizadas.

O QUE A DEMO PROVA

Sem compressão: qualidade cai de 0.95 para 0.45 em 10 turnos (-53%). Com compressão: qualidade se mantém acima de 0.85 mesmo em 10 turnos (-9%). A diferença é resultado de menos tokens de Observation no scratchpad — context rot reduzido.

Compressão do scratchpad e medição de impacto ★★★

Grupo 3–4 · 20 minutos · Google Colab

1. **Complete as 4 lacunas** — contar_tokens_scratchpad (concatenar action.log + obs), buscar_na_web_comprimida (invocar chain_resumir com a chave correta), 5 perguntas do domínio e os dois argumentos do plt.bar.
2. **Analise o gráfico** — em qual pergunta a diferença de tokens foi maior? Por quê? (Dica: resultado de busca web é maior que resultado do RAG)
3. **Calcule a redução média** em % de tokens usando a fórmula $(1 - \text{mean}(\text{tok_com})/\text{mean}(\text{tok_sem})) * 100$. Documente em célula markdown.

Gabarito das lacunas

Lacuna 1: texto += f"{action.log}\n{obs}\n" ; enc.encode(texto)

Lacuna 2: chain_resumir.invoke({"texto": bruto})

Lacuna 3: 5 perguntas reais do domínio do grupo

Lacuna 4: tok_sem (1º bar) ; tok_com (2º bar)



Desafio — memória episódica: implementar salvar_sessao() e lembrar_sesoes_anteriores como 4ª tool do agente. Salvar as 5 respostas do lab e depois perguntar ao agente "o que discutimos antes sobre [tema]?" — ele deve recuperar do ChromaDB episódico.

Referências

BLOG

Anthropic Engineering — "Context Engineering for AI Agents" (setembro 2025). Fonte primária desta aula — princípio da ação mínima, tipos de memória, curadoria de scratchpad. anthropic.com/engineering/context-engineering

DOCS

Model Context Protocol — Especificação oficial, servidores disponíveis e guia de implementação. modelcontextprotocol.io

PAPER

Liu, N. et al. — "Lost in the Middle: How Language Models Use Long Contexts." EMNLP, 2023. A base empírica do context rot em contextos longos. arxiv.org/abs/2307.03172

DOCS

LangChain MCP Adapters — Integrar servidores MCP como tools LangChain. github.com/langchain-ai/langchain-mcp-adapters

LIVRO

Russell, S.; Norvig, P. — Inteligência Artificial. 3ª ed. Pearson, 2016. Cap. 2 — Agentes e ambientes: a analogia memória=RAM / conhecimento=HD que fundamenta os 3 tipos de memória agêntica.

Curadoria que mantém a qualidade 🧠

Context rot medido, compressão implementada. O agente agora mantém qualidade em loops longos. Na Aula 11 tudo isso vira um produto com URL pública.

PRÓXIMA AULA — AULA 11 · 26/10

Aula Integradora — Agente com RAG + Gradio ao vivo

100% lab. Integrar tudo. Publicar URL pública. Entregar CKP03.

🚩 CKP03 — trazer na Aula 11

Notebooks de A08, A09 e A10 funcionando. ChromaDB do CKP02 disponível.